

1.

Which tag is used to define a clickable image map in HTML5?

- a) <map>
- b) <area>
- c)
- d) <figure>

Answer: a) <map>

2.

What is the effect of display: flex; justify-content: space-between;?

- a) Aligns children vertically with equal spacing
- b) Aligns children horizontally with space between them
- c) Centers children horizontally
- d) Stacks children in a column

Answer: b) Aligns children horizontally with space between them

3.

What is the output?

```
console.log(5 + "5");
```

- a) 10
- b) "55"
- c) 25
- d) Error

Answer: b) "55"

4.

Which method is used to add an event listener?

- a) addEvent()
- b) attachEvent()
- c) addEventListener()
- d) onEvent()

Answer: c) addEventListener()

5.

Which property keeps a background image fixed while scrolling?

- a) background-repeat: no-repeat
- b) background-attachment: fixed
- c) background-position: center
- d) background-size: cover

Answer: b) background-attachment: fixed

React

6.

What is the purpose of the useState hook?

- a) Fetch data from API
- b) Manage state in functional components
- c) Handle side effects
- d) Create context

Answer: b) Manage state in functional components

7.

What does this render?

```
return <h1>{undefined}</h1>;
```

- a) Nothing (empty string)
- b) "undefined"
- c) Error
- d) null

Answer: a) Nothing (empty string)

8.

Which hook performs side effects?

- a) useEffect
- b) useReducer

- c) useContext
- d) useCallback

Answer: a) useEffect

9.

How do you pass data from parent to child?

- a) Using state
- b) Using props
- c) Using context
- d) Using hooks

Answer: b) Using props

10.

Correct way to update state?

```
const [user, setUser] = useState({ name:"John", age:30 });
```

- a) setUser({ name:"Jane" })
- b) setUser({ ...user, name:"Jane" })
- c) user.name="Jane"
- d) setUser(user.name="Jane")

Answer: b) setUser({ ...user, name:"Jane" })

11.

Purpose of key prop in a list?

- a) Styling
- b) Identifying unique elements for efficient rendering
- c) Handling events
- d) Managing state

Answer: b) Identifying unique elements for efficient rendering

12.

Lifecycle method replaced by useEffect?

- a) componentDidMount
- b) render
- c) constructor
- d) setState

Answer: a) componentDidMount

13.

What does JSX stand for?

- a) JavaScript XML
- b) JavaScript Extension
- c) JSON XML
- d) Java Syntax

Answer: a) JavaScript XML

14.

How to create a ref in functional component?

- a) `const ref = useRef();`
- b) `const ref = createRef();`
- c) `const ref = useState();`
- d) `const ref = useEffect();`

Answer: a) `const ref = useRef();`

15.

If count = 0, output?

```
useEffect(() => {  
  setCount(count + 1);  
}, []);
```

- a) 0
- b) 1
- c) 2
- d) Infinite loop

Answer: b) 1

MongoDB

16.

Primary key in MongoDB document?

- a) _id
- b) id
- c) key
- d) index

Answer: a) _id

17.

Find users where age > 25?

- a) `db.users.find({ age: { $gt: 25 } })`
- b) `db.users.find({ age: { $gte: 25 } })`
- c) `db.users.find({ age: { $lt: 25 } })`
- d) `db.users.find({ age: 25 })`

Answer: a) `db.users.find({ age: { $gt: 25 } })`

18.

What does \$push do?

- a) Replace field
- b) Add element to array
- c) Remove element
- d) Update field

Answer: b) Add element to an array

19.

Which index improves query performance?

- a) Compound index
- b) Text index
- c) Unique index
- d) All of the above

Answer: d) All of the above

Angular

20.

Purpose of *ngFor?

- a) Conditional display
- b) Iterate list
- c) Data binding
- d) Form submission

Answer: b) Iterate over a list

21.

Decorator defining component?

- a) @Component
- b) @Directive
- c) @NgModule
- d) @Injectable

Answer: a) @Component

22.

Output:

```
<div *ngIf="false">Hello</div>
```

- a) Hello
- b) Nothing
- c) Error
- d) false

Answer: b) Nothing

23.

How to inject service?

- a) Using @Inject
- b) In constructor
- c) Using @Service
- d) In template

Answer: b) In constructor

24.

What does FormBuilder simplify?

- a) HTTP requests
- b) Reactive forms creation
- c) Template rendering
- d) Dependency injection

Answer: b) Reactive form creation

25.

Module for HTTP requests?

- a) HttpClientModule
- b) FormsModule
- c) RouterModule
- d) CommonModule

Answer: a) HttpClientModule

Core Java Code Snippets

26.

Output?

```
int x = 5;  
System.out.println(x++ + ++x);
```

- a) 10
- b) 11
- c) 12
- d) 13

Answer: c) 12

27.

```
String s1="Hello";  
String s2=new String("Hello");  
System.out.println(s1==s2);
```

- a) true
- b) false
- c) Error
- d) null

Answer: b) false

28.

```
List<Integer> list=new ArrayList<>();  
list.add(1);  
list.add(2);  
list.remove(Integer.valueOf(1));  
System.out.println(list);
```

- a) [1,2]
- b) [2]
- c) []
- d) Error

Answer: b) [2]

29.

```
static int x=10;  
Test t=new Test();  
t.x=20;  
System.out.println(Test.x);
```

- a) 10
- b) 20
- c) 30
- d) Error

Answer: b) 20

30.

```
int[] arr={1,2};  
System.out.println(arr[2]);
```

with try-catch.

- a) 1
- b) 2
- c) Error
- d) null

Answer: c) Error

31. Java: What is the purpose of the final keyword?

- a) Prevents method overriding
- b) Allows dynamic binding
- c) Enables garbage collection
- d) Permits multiple inheritance

Answer: a) Prevents method overriding

32. Java: Which collection is synchronized by default?

- a) ArrayList
- b) HashMap
- c) Vector
- d) LinkedList

Answer: c) Vector

33. Java: What is the default access modifier for class members?

- a) public
- b) private
- c) protected
- d) package-private

Answer: d) package-private

34. Java: What does the transient keyword do?

- a) Prevents serialization
- b) Enables multithreading
- c) Marks a variable as static
- d) Allows method overriding

Answer: a) Prevents serialization

35. Java: Which interface is used for sorting objects?

- a) Comparable
- b) Comparator
- c) Serializable
- d) Cloneable

Answer: a) Comparable

TypeScript

36. TypeScript: What is the purpose of interface in TypeScript?

- a) Define class behavior
- b) Specify object structure
- c) Create modules
- d) Handle exceptions

Answer: b) Specify object structure

37. TypeScript: What is the type of let x: any;?

- a) number
- b) string
- c) any
- d) void

Answer: c) any

38. TypeScript: What does this code do?

```
function greet(name: string): string {  
  return `Hello, ${name}`;  
}
```

- a) Throws a type error
- b) Returns a string
- c) Returns void
- d) Returns any

Answer: b) Returns a string

39. TypeScript: Which keyword extends a class?

- a) implements
- b) extends
- c) super
- d) interface

Answer: b) extends

40. TypeScript: How do you define an optional property?

- a) name: string?
- b) name?: string
- c) name: string | null
- d) name: string = null

Answer: b) name?: string

Spring / Spring Boot

41. Spring: Which annotation defines a Spring bean?

- a) @Bean
- b) @Component
- c) @Service
- d) All of the above

Answer: d) All of the above

42. Spring Boot: What does @SpringBootApplication include?

- a) @Configuration, @EnableAutoConfiguration, @ComponentScan
- b) @RestController, @EnableWebMvc
- c) @Repository, @Transactional
- d) @Autowired, @Qualifier

Answer: a) @Configuration, @EnableAutoConfiguration, @ComponentScan

43. Spring: What is the output of this code?

```
@Component
public class MyBean {
    public String getName() {
        return "Hello";
    }
}
```

```
@Autowired
MyBean bean;
```

```
System.out.println(bean.getName());
```

- a) Hello
- b) null
- c) Error
- d) MyBean

Answer: a) Hello

44. Spring Boot: Which dependency enables REST APIs?

- a) spring-boot-starter-web
- b) spring-boot-starter-data-jpa
- c) spring-boot-starter-security
- d) spring-boot-starter-test

Answer: a) spring-boot-starter-web

45. Spring: What is Dependency Injection?

- a) Creating objects manually
- b) Injecting dependencies into objects
- c) Managing database connections
- d) Handling HTTP requests

Answer: b) Injecting dependencies into objects

46. Spring Boot: How do you externalize configuration?

- a) Using application.properties
- b) Using @Configuration
- c) Using @Bean
- d) Using @Autowired

Answer: a) Using application.properties

47. Spring: Which annotation maps HTTP GET requests?

- a) @PostMapping
- b) @GetMapping
- c) @RequestMapping
- d) @PutMapping

Answer: b) @GetMapping

48. Spring Boot: What is the default embedded server?

- a) Jetty
- b) Tomcat
- c) Undertow
- d) Netty

Answer: b) Tomcat

49. Spring: What does @Transactional do?

- a) Manages database transactions
- b) Configures REST endpoints
- c) Injects dependencies
- d) Scans components

Answer: a) Manages database transactions

50. Spring Boot: How do you connect to MongoDB?

- a) spring-boot-starter-data-mongodb
- b) spring-boot-starter-data-jpa
- c) spring-boot-starter-web
- d) spring-boot-starter-security

Answer: a) spring-boot-starter-data-mongodb

DevOps

51. DevOps: What is the purpose of Docker?

- a) Code compilation
- b) Containerization
- c) Version control
- d) Load balancing

Answer: b) Containerization

52. DevOps: Which command builds a Docker image?

- a) docker run
- b) docker build
- c) docker push
- d) docker pull

Answer: b) docker build

53. DevOps: What is CI/CD?

- a) Continuous Integration / Continuous Deployment
- b) Code Inspection / Code Delivery
- c) Continuous Improvement / Continuous Development
- d) Code Integration / Code Deployment

Answer: a) Continuous Integration / Continuous Deployment

54. DevOps: Which tool manages container orchestration?

- a) Jenkins
- b) Kubernetes
- c) Git
- d) Ansible

Answer: b) Kubernetes

55. DevOps: What does nginx -g "daemon off;" do in a Docker container?

- a) Runs NGINX in the background
- b) Runs NGINX in the foreground
- c) Stops NGINX
- d) Restarts NGINX

Answer: b) Runs NGINX in the foreground

56. DevOps: Which file defines a Jenkins pipeline?

- a) Dockerfile
- b) Jenkinsfile
- c) pom.xml
- d) build.gradle

Answer: b) Jenkinsfile

57. DevOps: What is Infrastructure as Code?

- a) Manual server setup
- b) Defining infrastructure using scripts
- c) Database management
- d) Application deployment

Answer: b) Defining infrastructure using scripts

58. DevOps: Which AWS service manages containerized apps?

- a) EC2
- b) ECS
- c) S3
- d) RDS

Answer: b) ECS

Microservices

59. Microservices: What is a key benefit of microservices architecture?

- a) Monolithic codebase
- b) Independent scalability
- c) Tight coupling
- d) Single database

Answer: b) Independent scalability

60. Microservices: Which pattern ensures loose coupling in microservices?

- a) Circuit Breaker
- b) Service Registry
- c) Event-Driven
- d) Monolith

Answer: c) Event-Driven